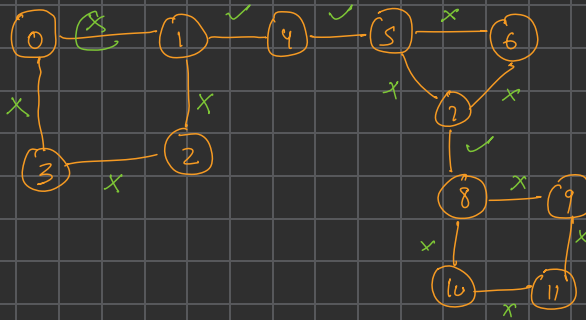
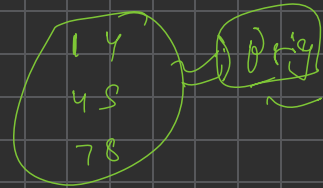
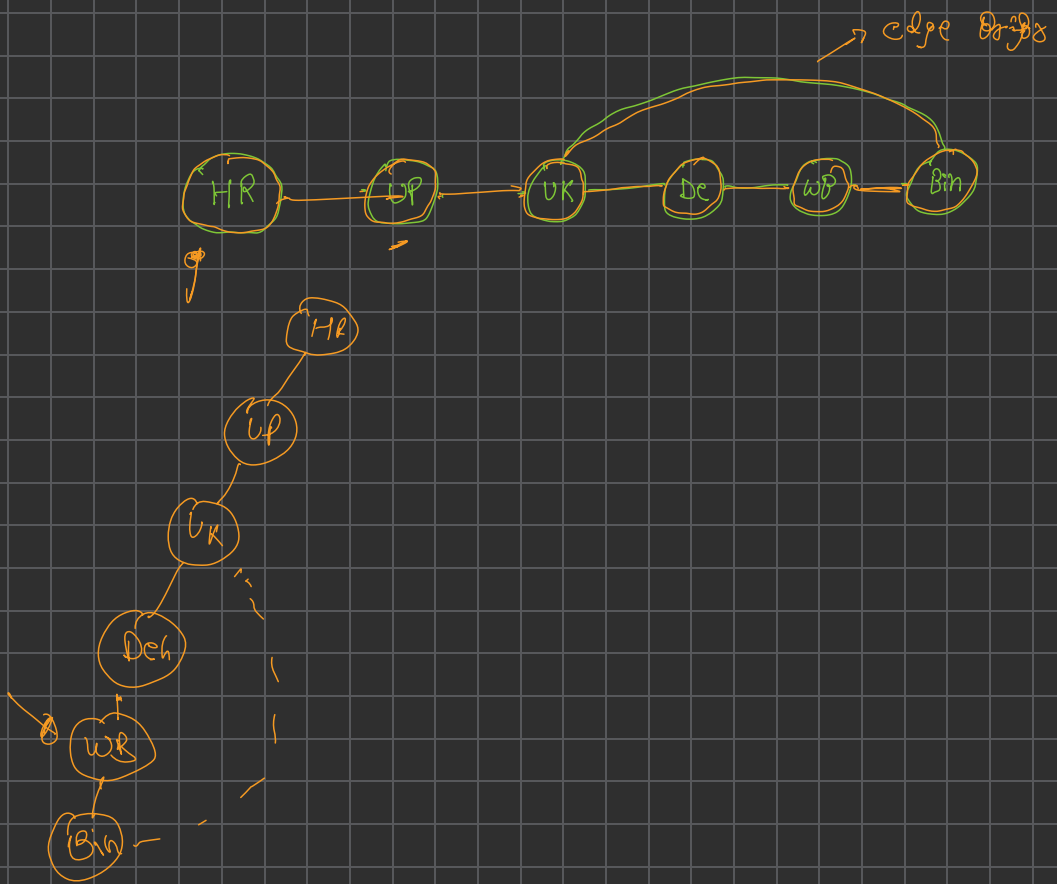


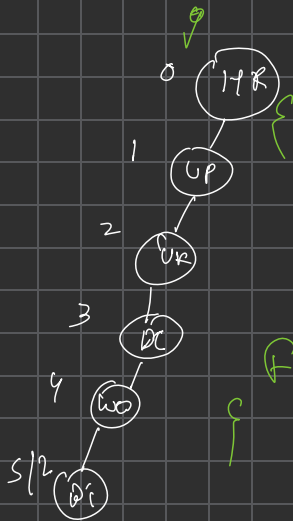
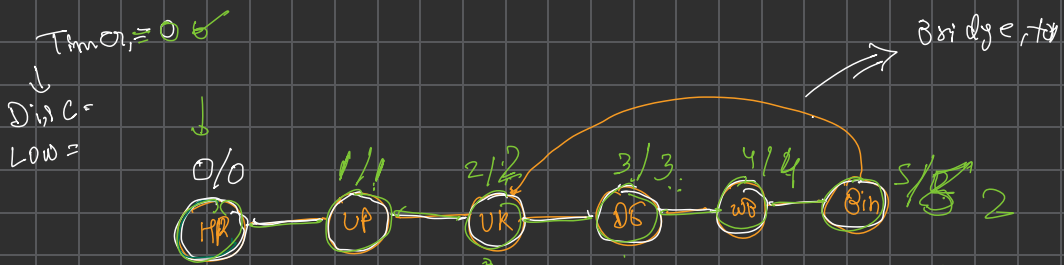


[Bridge in a graph]





$\&$ $disc[node] < low[neib]$
bridge hai



```

fn findBridge (
    Disc[Node] = Low[Node] = timer;
    visited[Node] = 1;
    timer++;

```

```

for (auto neib : adj[node])

```

```

    if (neib == parent)
        continue;

```

```

    else if (visited[neib])
        Low[node] = min(Low[neib], Low[node]);

```

```

    else

```

```

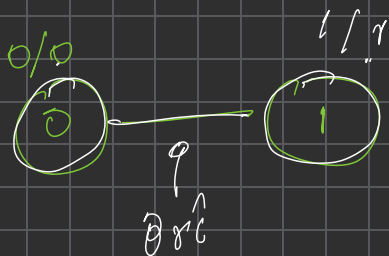
        findBridge(
            if (disc[node] < Low[neib])
                bridge++;

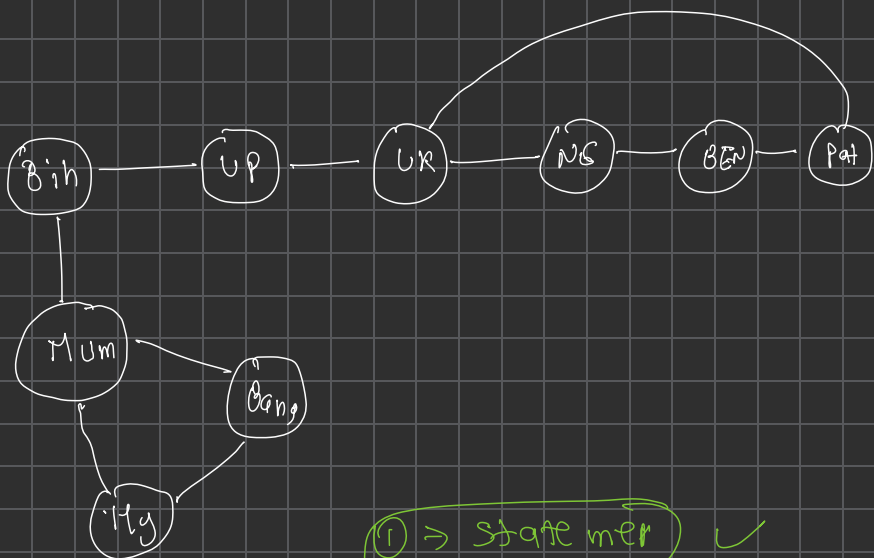
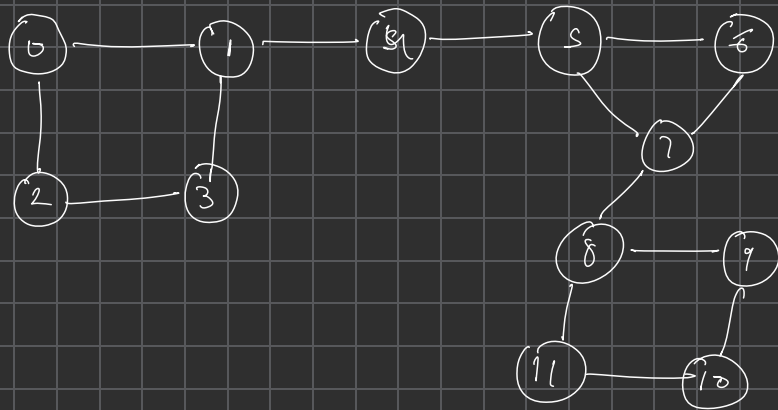
```

```

        Low[node] = min(Low[node], Low[neib]);
    }
}

```





① → state mer ✓

↳ use adjacency
⇒ activation

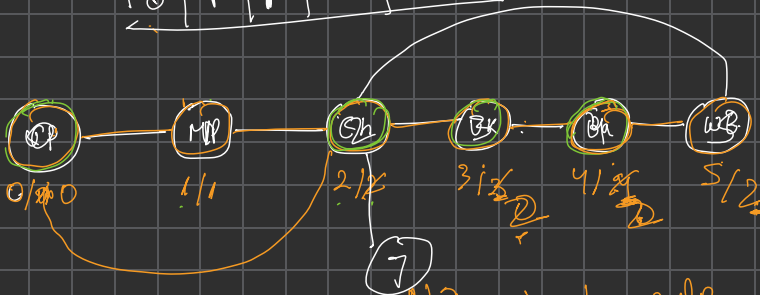
Edge case handle

array store

1 child

log

0	1	2	3	4	5
0	1	0	0	0	0



Articulation point

if [disc [node] <= Low [neib]

↳ I am articulation

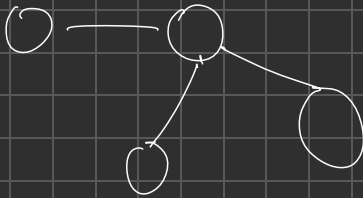
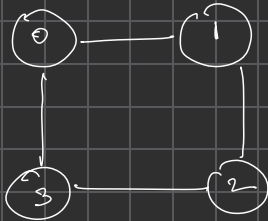
if [visited [neib]

low [node] = min [low [node], disc [neib]]

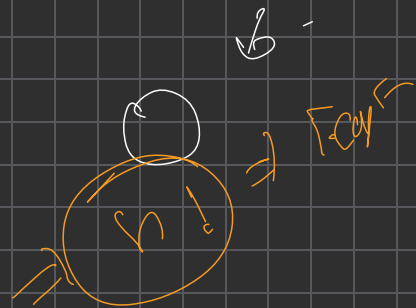
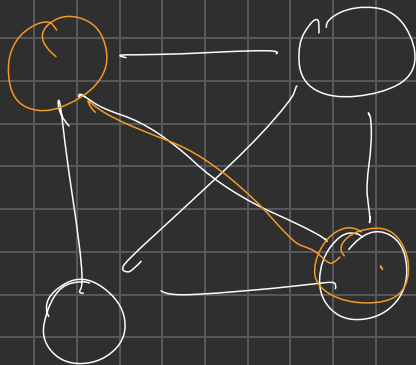


Hamiltonian

↓ node



You have to traverse all the node exactly one time



$$(n-1) + (n-2) + (n-3)$$

